26/06/26 **Deepak Kumar**

Artificial Neural Network (ANN)

1. Definition

Artificial Neural Network (ANN) Deep Learning ka ek model hai jo human brain ke neurons se inspired hota hai. Ye input data se patterns aur relationships seekhkar prediction ya classification karta hai.

Simple Definition

ANN ek computational model hai jo interconnected neurons ka use karke data se learn karta hai aur intelligent predictions karta hai.

---

2. ANN Architecture

```
Input Layer
      ↓
Hidden Layer(s)
      ↓
Output Layer
```

Components

- **Input Layer** → Features receive karta hai.
- **Hidden Layer** → Data processing aur pattern learning karti hai.
- **Output Layer** → Final prediction deti hai.

---

3. Advantages of ANN

Learns Complex Patterns

Non-linear relationships ko easily learn kar sakta hai.

High Accuracy

Large datasets par achha performance deta hai.

Automatic Feature Learning

Important features khud identify kar leta hai.

Adaptability

Different types ke problems solve kar sakta hai.

Fault Tolerance

Kuch noisy data hone par bhi kaam karta hai.

---

4. Disadvantages of ANN

Large Data Requirement

Achhi performance ke liye zyada data chahiye.

High Computational Cost

Training me zyada CPU/GPU resources lagte hain.

Time Consuming

Training process slow ho sakta hai.

Hyperparameter Tuning

Layers, neurons, learning rate choose karna difficult ho sakta hai.

Overfitting

Small dataset par overfit ho sakta hai.

---

5. Limitations of ANN

Black Box Model

Decision ka exact reason explain karna difficult hota hai.

Large Training Data Needed

Small datasets par hamesha best perform nahi karta.

High Hardware Requirement

Complex networks ke liye GPU ki need ho sakti hai.

Sensitive to Hyperparameters

Wrong architecture performance ko affect kar sakti hai.

---

6. Applications of ANN

Finance

- Loan Approval Prediction
- Credit Risk Analysis
- Fraud Detection

Healthcare

- Disease Prediction
- Medical Diagnosis

E-Commerce

- Recommendation Systems
- Customer Behavior Analysis

Transportation

- Traffic Prediction
- Self-Driving Cars

Speech Recognition

- Voice Assistants
- Speech-to-Text

Computer Vision

- Image Classification
- Face Recognition

-

ANN Architecture

```
Input Layer
      ↓
Hidden Layer(s)
      ↓
Output Layer
```

Input Layer

Receives input features from the dataset.

Hidden Layer

Learns patterns and relationships from data.

Output Layer

Produces the final prediction or result.

Interview Answer

What is ANN and why is it used?

Artificial Neural Network (ANN) is a Deep Learning model inspired by the human brain. It is used to learn complex patterns and relationships from data, especially when traditional machine learning algorithms are unable to achieve the desired accuracy. ANN is widely used in image recognition, speech recognition, recommendation systems, fraud detection, and predictive analytics.

ANN Kab Use Karte Hain?

1. Jab Dataset Bada Ho

Agar dataset me bahut saare records ho (thousands ya lakhs me), tab ANN achha perform karta hai.

Kyu?

Kyuki ANN zyada data se complex patterns ko achhi tarah learn kar pata hai.

---

2. Jab Problem Complex Ho

Agar input aur output ke beech relationship simple na ho aur bahut saare factors affect kar rahe ho.

Kyu?

Kyuki ANN hidden layers ki help se complex aur non-linear relationships ko learn kar sakta hai.

---

3. Jab High Accuracy Chahiye Ho

Jaise:

- Fraud Detection
- Loan Approval
- Disease Prediction
- Customer Churn Prediction

Kyu?

Kyuki ANN traditional ML models ke comparison me better accuracy de sakta hai.

---

4. Jab Pattern Recognition Karna Ho

Jaise:

- Image Recognition
- Face Recognition
- Handwriting Recognition
- Speech Recognition

Kyu?

Kyuki ANN automatically important patterns aur features ko identify kar leta hai.

---

5. Jab Traditional ML Achha Result Na De

Agar Logistic Regression, Decision Tree, Random Forest jaise models expected accuracy na de rahe ho.

Kyu?

Kyuki ANN deeper learning karke hidden patterns ko capture kar sakta hai.

ANN Kab Use Nahi Karte?

Small Dataset

Kyu?

ANN ko achha learn karne ke liye zyada data chahiye hota hai.

Simple Problems

Kyu?

Simple problems ko ML algorithms kam computation me solve kar dete hain.

Explainable Model Chahiye Ho

Kyu?

ANN ek **Black Box Model** hai, isliye prediction ka exact reason batana difficult hota hai.

Interview Answer (Short)

ANN tab use karte hain jab dataset bada ho, problem complex ho, non-linear relationships ho aur high accuracy required ho. ANN hidden layers ki madad se complex patterns ko learn karke better predictions karta hai.

□ ANN (Artificial Neural Network) me Use Hone Wale Important Algorithms aur Methods

1. Forward Propagation

- Input data ko network me pass kiya jata hai.
- Data Input Layer → Hidden Layer → Output Layer tak jata hai.

- Model prediction generate karta hai.
- Training ka pehla step hota hai.

---

2. Backpropagation

- Prediction aur Actual Value ke beech error calculate kiya jata hai.
- Error ko output layer se input layer ki taraf bheja jata hai.
- Weights aur biases update kiye jate hain.
- Model ko learn karne me help karta hai.

---

3. Gradient Descent

- Loss ko minimize karne ke liye use hota hai.
- Weights ko optimize karta hai.
- Best solution find karne ki technique hai.

Types of Gradient Descent

Batch Gradient Descent

- Pure dataset ko ek saath process karta hai.
- Slow ho sakta hai.

Stochastic Gradient Descent (SGD)

- Ek record par training karta hai.
- Fast hai lekin unstable ho sakta hai.

Mini-Batch Gradient Descent

- Chhote batches me training karta hai.
- Most commonly used method.

---

4. Optimizers

Optimizer weights ko update karne ka smart method hai.

SGD (Stochastic Gradient Descent)

- Basic optimizer.

- Simple weight update karta hai.

Momentum

- Previous updates ko consider karta hai.
- Learning ko fast banata hai.

RMSProp

- Learning rate ko automatically adjust karta hai.
- Deep networks me useful hai.

Adam Optimizer

- Sabse popular optimizer.
- Fast aur stable training deta hai.
- Industry me sabse zyada use hota hai.

---

5. Activation Functions

Activation Function decide karti hai ki neuron active hoga ya nahi.

ReLU (Rectified Linear Unit)

- Hidden layers me sabse zyada use hota hai.
- Fast aur efficient hai.
- Vanishing Gradient problem ko reduce karta hai.

Sigmoid

- Output ko 0 se 1 ke beech convert karta hai.
- Binary Classification me use hota hai.

Tanh

- Output ko -1 se +1 ke beech convert karta hai.
- Sigmoid se better centered output deta hai.

Softmax

- Multi-Class Classification me use hota hai.
- Har class ki probability calculate karta hai.

---

6. Loss Functions

Loss Function model ki error measure karti hai.

Binary Crossentropy

- Binary Classification ke liye.
- Example:
    - Loan Approval
    - Churn Prediction
    - Spam Detection

Sparse Categorical Crossentropy

- Multi-Class Classification ke liye.
- Example:
    - Digit Recognition
    - Image Classification

Mean Squared Error (MSE)

- Regression Problems ke liye.
- Example:
    - House Price Prediction
    - Sales Prediction

---

7. Regularization Techniques

Overfitting ko control karne ke liye use ki jati hain.

Dropout

- Random neurons ko temporarily disable karta hai.
- Overfitting reduce karta hai.

Early Stopping

- Validation Loss increase hone par training stop kar deta hai.
- Unnecessary training se bachata hai.

---

8. Evaluation Metrics

Model ki performance measure karne ke liye use hote hain.

Accuracy

- Kitni predictions sahi hui.

Precision

- Positive predictions me kitni predictions correct thi.

Recall

- Actual positive values me se kitni identify hui.

F1 Score

- Precision aur Recall ka balance.

---

9. ANN Training Process

1. Forward Propagation
2. Loss Calculation
3. Backpropagation
4. Gradient Descent
5. Weight Update
6. Next Epoch
7. Model Convergence

Most Important ANN Topics (Placement Focus)

- Forward Propagation
- Backpropagation
- Gradient Descent
- SGD
- Adam Optimizer
- ReLU
- Sigmoid
- Softmax
- Binary Crossentropy
- MSE
- Dropout
- Early Stopping

- Accuracy
- Precision
- Recall
- F1 Score

ANN Project Life Cycle

```
1. Problem Statement
        ↓
2. Data Collection
        ↓
3. Data Understanding (EDA)
        ↓
4. Data Preprocessing
        ↓
5. Feature Selection
        ↓
6. Train-Test Split
        ↓
7. Feature Scaling
        ↓
8. Build ANN Architecture
        ↓
9. Compile Model
        ↓
10. Train Model
        ↓
11. Evaluate Model
        ↓
12. Predictions
        ↓
13. Save Model
        ↓
14. Deployment (Streamlit)
```

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Load Dataset

```
dc = pd.read_csv("Bank_Personal_Loan_Modelling.csv")
```

Basic EDA

```
dc.head()

dc.shape

dc.info()

dc.describe()
```

Data Cleaning

Missing Values

```
dc.isnull().sum()
```

Duplicate Values

```
dc.duplicated().sum()
```

Feature Selection

Target Column:

```
y = dc["Personal.Loan"]
```

Input Features:

```
X = dc.drop("Personal.Loan", axis=1)
```

```
X = dc.drop(
   ["Personal.Loan", "ID", "ZIP.Code"],
   axis=1
)
```

Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

ANN Model Building

```
model = Sequential()

model.add(Dense(64, activation="relu", input_shape=(11,)))

model.add(Dense(32, activation="relu"))

model.add(Dense(16, activation="relu"))

model.add(Dense(1, activation="sigmoid"))
```

Compile Model

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

Train Model

```
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2
)
```

Evaluate Model

```
loss, accuracy = model.evaluate(
    X_test,
    y_test
)

print("Accuracy:", accuracy)
```

Prediction

```
y_pred = model.predict(X_test)

y_pred = (y_pred > 0.5).astype(int)
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(
```

```
    y_test,
    y_pred
)

print(cm)
```

Classification Report

```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        y_pred
    )
)
```

Save Model

```
model.save("loan_ann.keras")
```

Save Scaler

```
import joblib

joblib.dump(
    scaler,
    "scaler.pkl"
)
```
